



Security Review For The Gavel Protocol



Collaborative Audit Prepared For:
Lead Security Expert(s):

The Gavel Protocol

Oxadrii

Kvar

Date Audited:

March 9 - March 27, 2026

Introduction

The Gavel Protocol — Introduction

The Gavel Protocol is an oracle-free, auction-based lending protocol deployed on Arbitrum. It is designed around a single premise: that the interest rate on a collateralised loan should be discovered by a competitive market, not computed by a utilisation curve or quoted by an oracle.

Borrowers post an ERC-20 token as collateral and specify the loan terms they want – principal, duration, collateral amount, and loan token. Lenders then compete in a descending auction, bidding down the interest rate until the auction period ends. The winning rate is the rate at which the marginal lender is willing to fund that specific risk, at that specific moment, against that specific collateral package. No oracle is queried, no curve is consulted, and no governance parameter sets the price.

This mechanism is the protocol's core innovation, and everything else follows from it. Because each loan is independently priced by the market, the protocol does not require a price feed for the collateral asset: the implied valuation is embedded in the lender's willingness to fund. Because rates are discovered rather than derived, the resulting dataset – a continuous stream of market-cleared rates across durations, collateral ratios, and asset pairs – constitutes a genuine on-chain yield curve for each collateral/loan token pair.

The protocol supports both fungible (ERC-20) and non-fungible (ERC-721) collateral through parallel contract sets, allowing the same auction-based rate discovery to apply whether a borrower is posting a tokenised asset or a specific NFT.

Architecture

The system comprises two deployed layers:

1. **The core protocol** (`LoanProtocol`, `PositionNFT`, and the NFT-collateral variants `NFTLoanProtocol` and `NFTPositionNFT`) is permissionless, zero-fee, and open-source. It handles auction execution, collateral custody, loan lifecycle management, position tokenisation, and a secondary marketplace for trading active loan positions. Both borrower and lender sides of every loan are represented as transferable ERC-721 position tokens, enabling secondary-market liquidity without disturbing the underlying loan.
2. **The curation layer** (`ListingService`, `NFTListingService`) is a separate, admin-gated contract that determines which collateral assets and loan tokens are eligible for use in the core protocol. This is the commercial surface of the system and the only component with retained administrative privileges. The separation is deliberate: the protocol itself is neutral infrastructure, and the curation decision – which assets are safe and appropriate – is an explicit, auditable, off-chain judgement.

Scope of this audit

Sherlock's review covered the full core protocol and curation layer: the auction and loan lifecycle logic, the position NFT contracts, the marketplace for position transfers, and the listing service that gates asset eligibility. The audit surfaced 13 findings – 8 Medium, 4 Low/Informational, and 0 High or Critical – all of which have been resolved in the `audit-fixes` branch, with fixes applied under a minimum-diff discipline.

Sherlock additionally recommended a **minimum offer threshold on the marketplace** to mitigate spam offers on listed positions. This is a design improvement rather than an in-scope fix and will be addressed in a follow-up release prior to mainnet, alongside a protocol-initiated addition: a `MarkDefault` **event** to provide a clean on-chain signal of default state transitions, which is needed for downstream indexing by planned extensions to the Gavel ecosystem.

Trust model

With the fixes applied, the core protocol operates without oracles, without privileged rate-setting, and without governance over active loans. The only retained administrative surface is the listing service, which can add, remove, or adjust parameters for eligible assets but cannot affect loans already in progress. The protocol's approach to security is therefore to minimise the set of things that require trust in the first place, and to make the remaining trust assumptions – curation, and the integrity of the listed token contracts themselves – explicit and narrow.

Scope

Repository: [JamieFrame/The-Gavel-Audit](#)

Audited Commit: `373076e67b667d1e5e08b2b89440547d69572671`

Final Commit: `c21f085d221ab364822c32fe64b8b4985d9c2384`

Files:

- `contracts/interfaces/ILoanProtocol.sol`
- `contracts/interfaces/INFTLoanProtocol.sol`
- `contracts/interfaces/INFTPositionNFT.sol`
- `contracts/interfaces/IPositionNFT.sol`
- `contracts/ListingService.sol`
- `contracts/LoanProtocol.sol`
- `contracts/NFTListingService.sol`
- `contracts/NFTLoanProtocol.sol`
- `contracts/NFTPositionNFT.sol`

- contracts/PositionNFT.sol

Final Commit Hash

c21f085d221ab364822c32fe64b8b4985d9c2384

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	9	4

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue M-1: Collateral will stay in the old contract if `setLoanProtocol()` is used [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-02-the-gavel-protocol-mar-9th/issues/11>

Vulnerability Detail

If `setLoanProtocol()` is used to replace `LoanProtocol`, loan accounting might be migrated depending on the new implementation, but collateral stays in the old contract with the current `LoanProtocol` implementation. The new contract won't have those funds, so repayments/claims can't complete unless state and funds are migrated too.

Also the new contract must continue `auctionId` sequencing from the old one, otherwise position IDs will collide or point to the wrong loans. However, that's entirely dependent on the new implementation.

Recommendation

Explained above

Issue M-2: NFTLoanProtocol::createAuction() allows bidStep to be less than MIN_BID_STEP if it's not 0 [RE-SOLVED]

Source:

<https://github.com/sherlock-audit/2026-02-the-gavel-protocol-mar-9th/issues/12>

Vulnerability Detail

Right now `MIN_BID_STEP == 1`, but it's expected that this will be set to some meaningful value. `LoanProtocol::createAuction()` correctly sets `bidStep` to `MIN_BID_STEP` if it's below that. However, `NFTLoanProtocol::createAuction()` allows `bidStep` to be less than `MIN_BID_STEP` if it's not 0.

Recommendation

Do not allow `bidStep` to be less than `MIN_BID_STEP`.

Issue M-3: An attacker can DoS makeMarketplaceOffer() for a given loanId by repeatedly creating and canceling offers [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-02-the-gavel-protocol-mar-9th/issues/13>

Vulnerability Detail

An attacker can DoS makeMarketplaceOffer() for a given loanId by repeatedly creating and canceling offers until marketplaceOfferNonce[loanId] reaches MAX_OFFERS_PER_LISTING. Because the nonce is monotonic and canceled offers still consume IDs, the check

```
if (marketplaceOfferNonce[loanId] >= MAX_OFFERS_PER_LISTING) revert TooManyOffers();
```

eventually blocks all future offers for that loan until nonce is reset.

Recommendation

Deduct the cancelled offers in the check.

Issue M-4: makeMarketplaceOffer() has no expectedPaymentToken check [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-02-the-gavel-protocol-mar-9th/issues/14>

Vulnerability Detail

In the updated code `buyPosition()` now uses `maxPrice` and `expectedPaymentToken`. However, the same issue still exists in `makeMarketplaceOffer()`.

`makeMarketplaceOffer()` has no `expectedPaymentToken`. A buyer can make an offer expecting to get charged token A, but if the seller unlists and re-lists with token B (more expensive), the buyer will make the offer with token B, even though he wanted to do it with token A. That offer can be accepted in the unexpected token if the seller accepts before buyer cancels.

Recommendation

Add `expectedPaymentToken` check in `makeMarketplaceOffer()`.

Issue M-5: ListingService should have a min bid step defined when a token is whitelisted [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-02-the-gavel-protocol-mar-9th/issues/15>

Vulnerability Detail

MIN_BID_STEP in LoanProtocol is currently a single fixed value used for every token. However, tokens have different prices and decimals, so using the same step for all of them doesn't make sense.

For example, WBTC has 8 decimals and costs around \$70k, meaning 1 wei $\approx$ 0.0007. Meanwhile, forexample, DAI has 18 decimals and is 1, so 1 wei $\approx$ \$0.000000000000000001.

Because of this, a single MIN_BID_STEP value will have very different economic effects depending on the token. The bid step should be token-specific (or normalized based on decimals / price) rather than using the same constant for every token.

ListingService should have a min bid step defined when a token is whitelisted. However, it does not in the current implementation.

Issue M-6: Borrower can create an auction with a very large bidStep, which will bypass the step logic [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-02-the-gavel-protocol-mar-9th/issues/16>

Vulnerability Detail

Right now `MIN_BID_STEP == 1`, but it's expected that this will be set to some meaningful value. However, a borrower can create an auction with a very large `bidStep`, and this can effectively bypass the step logic.

In `placeBid()` it calculates:

```
uint256 maxValidBid = auction.currentBid > auction.bidStep
    ? auction.currentBid - auction.bidStep
    : 0;
```

If `maxValidBid` becomes less than `auction.loanAmount`, it only checks:

```
if (maxValidBid < auction.loanAmount) {
    if (repaymentAmount >= auction.currentBid) revert BidTooHigh();
}
```

Here the `bidStep` requirement is basically ignored, and the only condition becomes:

`loanAmount < repaymentAmount < currentBid`

So bidders can lower the bid by any tiny amount.

Recommendation

Adding a max bid step value is a good fix here. However, it should be a % of `loanAmount`.

Issue M-7: repayLoan() transfers collateral to the borrower, instead of the collateral provider [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-02-the-gavel-protocol-mar-9th/issues/17>

Vulnerability Detail

Collateral should always go back to `auction.collateralFrom`, however in `repayLoan()` it's sent to `msg.sender`. Since the borrower calls `repayLoan()`, the collateral ends up going to them instead of the collateral provider.

Recommendation

Transfer the collateral to the collateral provider instead.

Issue M-8: LoanProtocol time-based flows don't account for the contract being paused [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-02-the-gavel-protocol-mar-9th/issues/18>

Vulnerability Detail

The `LoanProtocol` pause logic is very simple and the time-based flows don't account for the contract being paused. Consider this:

1. Alice creates an auction with low `maxRepayment` relative to `loanAmount` and provides a large `collateralAmount` to make it attractive.
2. Alice finalizes the auction and takes a loan from Bob.
3. Owner pauses `LoanProtocol` due to something unrelated.
4. `repayLoan()` is `whenNotPaused`, so Alice cannot repay.
5. The pause lasts longer than `GRACE_PERIOD`.
6. After unpaused, Alice can't repay anymore because `GRACE_PERIOD` expired, and Bob can call `claimCollateral()`, taking the large collateral Alice intended to repay.

Recommendation

Time-based flows should be paused too if the protocol is paused.

Issue M-9: marketplaceListingFee decimal assumption causes fee miscollection for non-USDC payment tokens [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-02-the-gavel-protocol-mar-9th/issues/22>

Summary

marketplaceListingFee is enforced against MAX_MARKETPLACE_LISTING_FEE = 100_000_000, a value that assumes 6-decimal tokens. However, positions can be listed using any paymentToken, including 18-decimal tokens. Since the same flat fee value is applied regardless of the token's decimals, the protocol significantly under-collects fees when 18-decimal tokens are used, and the per-token fee amount is economically meaningless.

Vulnerability Detail

MAX_MARKETPLACE_LISTING_FEE is defined as:

```
uint256 public constant MAX_MARKETPLACE_LISTING_FEE = 100_000_000; // <@ $100 max
```

This hardcodes the assumption that the payment token has 6 decimals . When setMarketplaceListingFee is called, the fee is bounded against this cap:

```
function setMarketplaceListingFee(uint256 _fee) external onlyOwner {
    if (_fee > MAX_MARKETPLACE_LISTING_FEE) revert FeeTooHigh();
    ...
    marketplaceListingFee = _fee;
}
```

But listPositionWithFee accepts any paymentToken chosen by the seller:

```
function listPositionWithFee(
    uint256 loanId,
    string calldata positionType,
    address paymentToken, // <@ any token
    uint256 askingPrice
) external whenNotPaused nonReentrant {
    if (marketplaceListingFee > 0) {
        IERC20(paymentToken).safeTransferFrom(msg.sender, treasury,
            ↪ marketplaceListingFee);
        ...
    }
}
```

```
...  
}
```

If `marketplaceListingFee` is set to `5_000_000` (intended as \$5 USDC), a seller who uses an 18-decimal token would only pay `5_000_000` wei of that token (essentially zero).

The same decimal mismatch applies to `auctionFeeBps`: the global BPS value is applied to `collateralAmount` irrespective of that token's decimals, and fees in `accumulatedFees[token]` are tracked without any normalization.

Impact

Medium. The protocol fails to collect meaningful marketplace listing fees from sellers who choose 18-decimal payment tokens.

Code Snippet

<https://github.com/sherlock-audit/2026-02-the-gavel-protocol-mar-9th/blob/main/The-Gavel-Audit/contracts/ListingService.sol#L483>

Tool Used

Manual Review

Recommendation

Track fees and their caps per payment token, rather than globally.

Issue L-1: ListingService::withdrawFees() and accumulatedFees logic overall is useless in the current implementation

Source:

<https://github.com/sherlock-audit/2026-02-the-gavel-protocol-mar-9th/issues/19>

Vulnerability Detail

ListingService::withdrawFees() and accumulatedFees logic overall is useless in the current implementation, because accumulatedFees[token] is never incremented.

Issue L-2: ListingService::createListedAuction() calculates fee differently from calculateAuctionFee() [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-02-the-gavel-protocol-mar-9th/issues/20>

Vulnerability Detail

In ListingService::createListedAuction() the fee is calculated using collateralAmount:

```
uint256 fee = (collateralAmount * auctionFeeBps) / 10000;
```

However, in calculateAuctionFee() the input is loanAmount. Also the natspec says:

```
/// @notice Calculate auction fee for a given loan amount  
/// @param loanAmount Principal amount
```

Issue L-3: Auction would make sense to end early if `repaymentAmount == auction.loanAmount` [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-02-the-gavel-protocol-mar-9th/issues/21>

Vulnerability Detail

In `placeBid()`, if `repaymentAmount == auction.loanAmount`, no better bid can exist. The auction still runs until `block.timestamp > auction.auctionEnd`, but in this case that extra time is pointless. This scenario is possible when collateral is high relative to the loan amount, so someone is willing to provide a loan with zero interest.

Issue L-4: Auction listing fee is charged before borrower receives loan funds

Source:

<https://github.com/sherlock-audit/2026-02-the-gavel-protocol-mar-9th/issues/23>

Summary

In `createListedAuction`, the auction listing fee is deducted from the borrower's wallet before the auction is finalized and the loan is disbursed. At the time of auction creation, the borrower has not yet received any of the `loanToken` they are attempting to borrow. This forces borrowers to front the fee from pre-existing funds, which may not be available, especially if their goal is to borrow that exact token.

Vulnerability Detail

`createListedAuction` computes and immediately transfers the fee in `collateralToken` from the borrower's wallet:

```
function createListedAuction(...) external whenNotPaused nonReentrant returns
↳ (uint256 auctionId) {
    ...
    // Calculate and collect auction listing fee (in collateral token from user's
    ↳ wallet)
    uint256 fee = (collateralAmount * auctionFeeBps) / 10000;
    if (fee > 0) {
        // Transfer fee from borrower's wallet to treasury
        IERC20(collateralToken).safeTransferFrom(msg.sender, treasury, fee);
    }

    // Create the auction - full collateralAmount goes to the auction
    auctionId = loanProtocol.createAuctionFor(...);
    ...
}
```

The fee is taken at auction creation time, long before any lender bids and before the auction is finalized. Since the fee is charged in `collateralToken`, the borrower must hold extra `collateralToken` in their wallet on top of what they already deposited as collateral.

Impact

Low, fees are incorrectly charged prior from obtaining the loan, leading to a potential barrier entry for users not holding the expected borrowed token.

Code Snippet

<https://github.com/sherlock-audit/2026-02-the-gavel-protocol-mar-9th/blob/main/The-Gavel-Audit/contracts/ListingService.sol#L428>

Tool Used

Manual Review

Recommendation

Collect the auction listing fee at the time the auction is **finalized** , after the borrower receives funds.

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.